

Veillez également à terminer la mise en place de votre dépôt `gitlab`. Notez de plus que toutes les classes à développer dans ce travail sont à regrouper au sein d'un même paquetage `tp03`. Il en sera de même pour tous les TPs à venir, chacun dans son propre paquetage.

Exercice 1 : Manipulation d'énumérations et de méthodes statiques

Q1. On souhaite bénéficier d'une classe `Card`, mais qui, au lieu de reposer sur des entiers comme vous avez pu le faire en TD, va reposer sur des énumérations. On rappelle les méthodes liées aux énumérations :

enum E	
int	<code>compareTo(E o)</code> Compares this enum with the specified object for order.
boolean	<code>equals(Object other)</code> Returns true if the specified object is equal to this enum constant.
String	<code>name()</code> Returns the name of this enum constant, exactly as declared in its enum declaration.
int	<code>ordinal()</code> Returns the ordinal of this enumeration constant (its position in its enum declaration, where the initial constant is assigned an ordinal of zero).
String	<code>toString()</code> Returns the name of this enum constant, as contained in the declaration.
static E	<code>valueOf(String name)</code> Returns the enum constant of the specified name.
static E[]	<code>values()</code> Returns an array containing all possible values defined by the enum.

Q1.1. Récupérez les énumérations `Color` et `Rank` et intégrez-les à votre projet. Leur code est le suivant :

```
public enum Color {
    CLUB, DIAMOND, HEART, SPADE;
}

public enum Rank {
    SEVEN, EIGHT, NINE, TEN, JACK, QUEEN, KING, ACE;
}
```

Q1.2. Écrivez la classe `Card` pour satisfaire la définition suivante :

Card
- color : Color - rank : Rank
+ Card(Color, Rank) + Card(String, String) + getColor() : Color + getRank() : Rank + equals(Card) : boolean + toString() : String

Q1.3. Récupérez la classe `UseCard`, intégrez-la à votre projet afin de valider votre méthode `equals`.

Q2. Intéressons-nous à la classe `LocalDate` qui permet de manipuler des dates facilement.

Q2.1. Parcourez la javadoc associée à la classe `LocalDate` afin de bien identifier les méthodes qui sont statiques, ainsi que celles permettant, par exemple d'instancier une date, de comparer chronologiquement deux dates, d'afficher une date, de calculer l'écart entre deux dates, de retourner l'année, le mois, le jour ou le nombre de jours dans le mois. . .

Q2.2. Écrivez au sein d'une classe `UseLocalDate`, une méthode `main` créant deux dates `d1` et `d2`, correspondant respectivement à la date d'aujourd'hui et la date de votre anniversaire, et les affiche.

Q2.3. Ajoutez les instructions nécessaires à la création d'une date `d3` aléatoire dans les 30 dernières années et l'affiche.

Q2.4. Affichez ensuite la date la plus ancienne entre `d2` et `d3`. Cherchez dans la documentation la ou les méthodes appropriées.

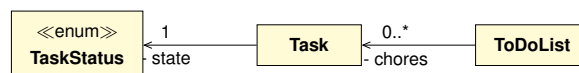
Q2.5. Affichez la date 7 jours après la date d'aujourd'hui, via l'ajout d'une durée et la méthode `plusDays`.

Q2.6. Calculez l'écart en année entre les dates `d2` et `d1`, par la méthode `until`. L'unité d'écart se spécifie grâce à l'énumération `ChronoUnit` (du paquetage `java.time.temporal`) décrivant les différentes unités utilisables.

Q3. Réalisez un `commit` de votre travail associé au message : `"tp00-03::exo-static"`.

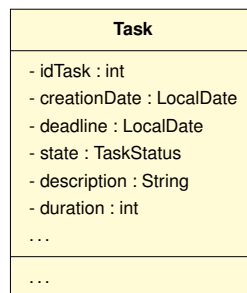
Exercice 2 : La classe Task

On souhaite implémenter une tâche comme celles qu'on retrouve dans une *"ToDo list"*, une liste de tâches à effectuer. Chaque tâche est caractérisée, entre autres, par une date de création, une date limite, une durée et un statut. On peut ainsi créer une tâche nécessitant 5 journées de travail mais à rendre pour dans 2 semaines. Le programme respectera la structure suivante :



Q1. Écrivez l'énumération `TaskStatus`, permettant de définir les valeurs `TODO`, `ONGOING`, `DELAYED` ou `FINISHED`.

Q2. Écrivez la classe `Task`, caractérisée par un numéro d'identification, une date de création ainsi qu'une date limite, l'état d'avancement, une description ainsi qu'une durée. Ajoutez les *getters* uniquement.



Q3. Munissez cette classe de deux constructeurs différents, respectant les contraintes ainsi que les signatures suivantes :

```
Task(String description, int duration)
```

```
Task(String description, LocalDate creation, LocalDate deadline, int duration)
```

Seules la durée et la description sont essentielles. Par défaut, une tâche est créée à la date du jour courant pour une date limite fixée par défaut à 10 jours plus tard. Les tâches nouvellement créées ont toujours le statut `TODO`. Enfin, le numéro d'identification est un numéro automatique (un numéro d'ordre).

Q4. Écrivez une méthode `toString`, représentant une tâche sous une forme textuelle dans laquelle on retrouve l'ordre le numéro de tâche, sa description, son état, sa durée ainsi que sa date limite. La représentation doit respecter la forme suivante : `"T<id> = <description>:<status>(<duration>):<deadline>"`.

Q5. Adjoignez les méthodes nécessaires à la modification du statut d'une tâche, en sachant qu'on souhaite pouvoir le faire de deux manières : sans rien préciser (on change le statut pour le suivant, dans l'ordre de définition de l'énumération) ou en précisant un statut spécifique.

```
void changeStatus()
```

```
void changeStatus(TaskStatus st)
```

Q6. Écrivez une classe `UseTask` dont le programme principal réalise le scénario suivant :

- création d'une tâche `t1` intitulée *"finir exo1"*, qui doit durer 1 journée ;
- création d'une tâche `t2` intitulée *"finir exo2"*, créée le 01/02/2026 pour le 01/03/2026 avec une durée de 2 journées ;
- afficher les tâches ;
- changer le status de `t1` ;

- changer le status de `t2` pour `FINISHED` ;
 - afficher une nouvelle fois les tâches.
- N'oubliez pas de vérifier la cohérence de l'exécution.

Q7. Écrivez une méthode `isLate` déterminant si une tâche est en retard par rapport à sa date limite. Veillez à ce qu'une tâche achevée ne soit jamais considérée comme en retard.

```
boolean isLate()
```

Q8. Ajoutez une méthode `delay` permettant de repousser la date limite d'une tâche d'un nombre de jours donné.

```
void delay(int nbDays)
```

Q9. Complétez votre classe `UseTask` avec le scénario suivant :

- création d'une tâche `t3` intitulée "*finir exo3*", créée le 01/02/2026 pour le 10/02/2026 avec une durée de 5 demi-journées ;
 - création d'une tâche `t4` intitulée "*finir exo4*", créée le 01/02/2026 pour le 10/02/2026 avec une durée de 5 demi-journées ;
 - afficher les tâches et le fait qu'elles soient en retard ou non ;
 - changer le status de `t3` pour `FINISHED` ;
 - accorder un délai de 30 jour à `t4` ;
 - afficher une nouvelle fois les tâches et leur éventuel retard.
- N'oubliez pas de vérifier la cohérence de l'exécution.

Q10. Réalisez un `commit` de votre travail associé au message : "`tp00-03::exo-Task`".